# License Plate Detection System using Jetson Nano

## Concepts and Application

Gautam Ramesh, 7027187
Siddhanth Sharma, 7027189
Shubhankar Dumka, 7027187

1. Januar 2026

1. Introduction
2. Description of Application
3. Challenges and Solutions
4. Results
5. References

# Introduction

# Introduction I

The project focuses on designing a cost-effective, single-camera car license plate recognition system.

Traditional automatic number plate recognition systems rely on expensive hardware and centralized processing [Nat24].

The system enables license plate detection using compact deep-learning models deployed on embedded devices. [Tan+22] [JCQ23; Cor21].

Application: High vehicle throughput such as airport departures and arrivals, where manual verification is inefficient and error-prone. Focused on automated entry/exit recognition and dwell-time-based billing.

# Description of Application

# Description of Application I

**Motivation (Airport use case):** Departure terminals face congestion when drop-off vehicles overstay, causing queues and delays for travelers.

**Current workflow:** Entry booth issues a paper slip with timestamp; exit booth manually checks if stay exceeded **10 minutes** and collects a fee if needed.

**Goal:** Replace manual checks with an **Automatic Number Plate Recognition (ANPR)** system that logs *entry/exit time* per vehicle and automates barrier + fee decision.

**Why Jetson Nano:** Enable on-site, low-latency inference on embedded hardware (no heavy centralized processing) using compact deep-learning detectors [Tan+22; JCQ23; Cor21].

**Core tasks:** plate detection (YOLO) → crop/enhance (OpenCV) → OCR → validation → database + decision display [Nat24].

# Description of Application II

**Deployment:** Two checkpoints at an airport terminal: **Entry Gate** and **Exit Gate**, each with a camera + display + barrier.

**Entry Gate:** Detect license plate → store (`plate_id`, `entry_timestamp`) in a central database → open barrier.

**Exit Gate:** Detect license plate → fetch `entry_timestamp` → compute dwell time.

**Decision rule:** if dwell time ≤ **10 minutes** → show "OK / free exit"; else → show fee + request payment → open barrier after payment.

**Audit trail:** Write `exit_timestamp` (and optionally fee/payment status) for traffic analysis and enforcement reporting.

*Next slide shows the full system diagram and data flow.*

# Description of Application III

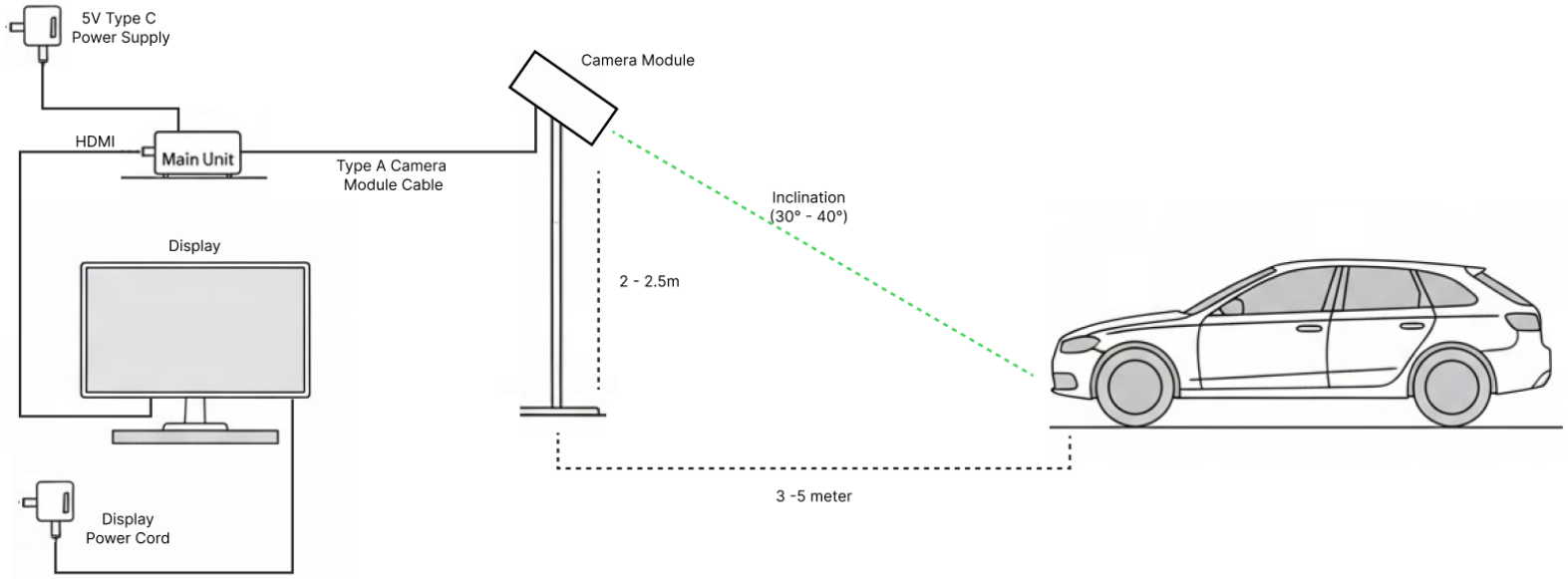


Abbildung: License Plate Detection System.

# Challenges and Solutions

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

# Challenge 1: System Memory Exhaustion

| The Problem | The Engineered Solution |
| --- | --- |
| **Conflict: Unified Memory Bottleneck**<br>The Jetson Nano shares a single 4GB RAM pool between the CPU and GPU.<br><br>• **OS Overhead:** Ubuntu 18.04 consumes ≈ 1.2 GB.<br>• **AI Load:** YOLOv5 + EasyOCR required ≈ 3.3 GB during initialization.<br><br>*Impact:* Demand (4.5 GB) exceeded physical limits. The Linux kernel invoked the ÖOM Killeränd terminated the process instantly. | **Solution: Hybrid Memory Architecture**<br>We engineered a split-resource strategy:<br><br>• **Virtual Expansion:** Configured a **6GB Swap Partition** on an external SSD to offload inactive OS pages, keeping physical RAM free for inference.<br>• **Compute Isolation:** Forced OCR to run on the **CPU** (`gpu=False`). This reserves the limited GPU VRAM entirely for YOLO detection. |

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

# Challenge 2: Legacy Software Ecosystem

| The Problem | The Engineered Solution |
| --- | --- |
| **Conflict: Dependency Hell**<br>The Jetson Nano relies on JetPack 4.6 (Ubuntu 18.04), which locks the environment to **Python 3.6**.<br><br>• **Incompatibility:** State-of-the-art libraries (Ultralytics YOLOv8, Pandas 2.0) require Python 3.8+ and modern C++ compilers, causing installation failures.<br>• **Deprecated APIs:** The `Pillow` image library recently removed attributes (like `ANTIALIAS`) used by older Jetson inference tools, causing runtime crashes. | **Solution: Cross-Compilation Workflow**<br>We bypassed the OS limitations using an external pipeline:<br><br>• **TensorRT Export:** Instead of running PyTorch natively, we trained the model on a PC, exported it to **ONNX**, and cross-compiled it into a **.engine** binary specifically optimized for the Nano's Maxwell GPU.<br>• **Monkey Patching:** We developed a custom initialization script that injects code at runtime to manually map old API calls to their new locations, preventing crashes in legacy libraries. |

# Challenge 3: Remote Development Breakdown

| The Problem | The Engineered Solution |
| --- | --- |
| **Conflict: glibc Version Lock**<br>In 2024, Microsoft updated the VS Code Remote Server to require **glibc 2.28** or higher.<br><br>• **The Lockout:** The Jetson Nano's OS is pinned to **glibc 2.27**.<br>• **Impact:** Standard SSH connections failed with ÖS Not Supportedërrors, blocking all direct code editing and file transfer capabilities on the device. | **Solution: SSH-FS Protocol Switch**<br>We architected a workaround that avoids running binary agents on the Jetson:<br><br>• **SFTP Mounting:** We switched to the **SSH FS** extension, which mounts the Jetson's external SSD ('/media/myssd') as a local network drive on the development laptop.<br>• **Benefit:** This protocol relies purely on standard SFTP, which does not check the glibc version, allowing full real-time coding access despite the OS obsolescence. |

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

# Challenge 4: Latency & Logic

| The Problem | The Engineered Solution |
| --- | --- |
| **Conflict: Hardware Constraints**<br>• **Latency:** Running the heavy Optical Character Recognition (OCR) model on every video frame dropped performance to an unusable **1 FPS** due to CPU bottlenecks.<br>• **Single-Camera Limit:** A standard parking system uses two cameras (Entry/Exit). With only one camera, the system lacks a physical sensor trigger to determine when a car has left the frame. | **Solution: State Machine Heuristics**<br>We developed a specialized algorithm to compensate:<br><br>• **Cooldown Timer:** We implemented a logic gate that pauses the OCR engine for **3 seconds** after a successful read. This keeps the video smooth by running only the lightweight Detector during the cooldown.<br>• **Last-SeenÄlgorithm:** The system updates a timestamp for every frame the car is visible. The billing logic defines Ëxit Timeäs the exact moment the timestamp ceases to update. |

# Results

# Results

The Apriori Algorithm enables mining datasets for frequent items.

Although it has a few limitations, it is widely used across multiple industries to gather analysis on customer behavior and enable decision making processes. Thus, allowing it to be an indispensable tool at an analyst's disposal.

University of Applied Sciences
HOCHSCHULE EMDEN·LEER

## Thanks a lot

for your attention

# References

# References I

[Cor21] A. Cortés. "Edge AI for Real-Time Computer Vision Applications". In: *Proceedings of the International Conference on Embedded Systems*. 2021, S. 88–95.

[JCQ23] Glenn Jocher, Ayush Chaurasia und Jing Qiu. *Ultralytics YOLOv8*. 2023. URL: https://github.com/ultralytics/ultralytics.

[Nat24] National Airports Council. *Vehicle Access Management Challenges in Modern Airports*. Techn. Ber. National Airports Council, 2024.

[Tan+22] Y. Tan u. a. "A Survey on Automated Number Plate Recognition Systems". In: *IEEE Access* 10 (2022), S. 12345–12360. DOI: 10.1109/ACCESS.2022.XXXXXXX.